

**UNIVERSIDADE DO VALE DO ITAJAÍ, CAMPUS ITAJAÍ
ENGENHARIA DE COMPUTAÇÃO**

LARISSA DE SOUSA GOUVEA

THIAGO YUKIO HORITA PACHECO

Trabalho M3 STR

ITAJAÍ
2024

Parte 1:

Teste oslat

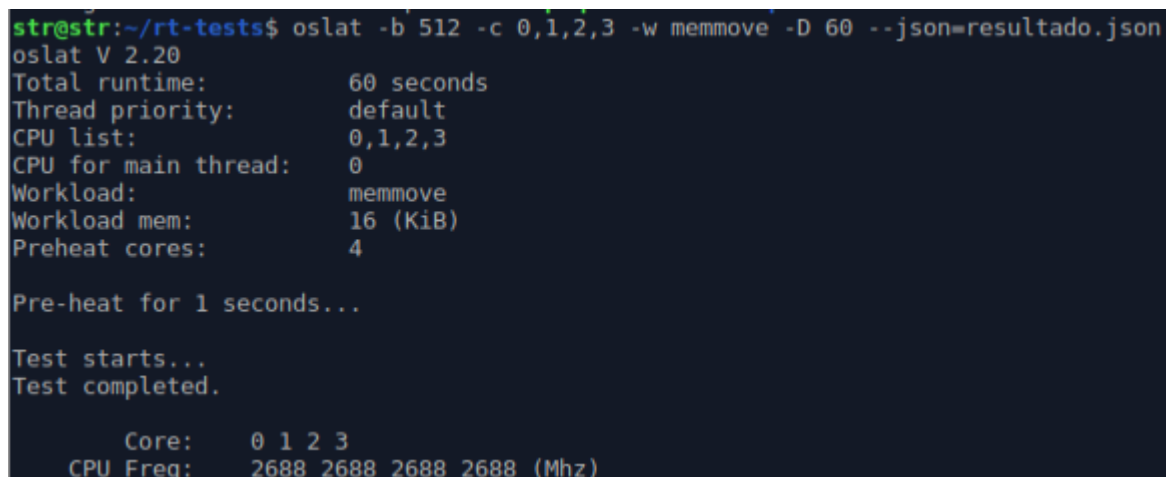
O **oslat** é um programa que opera em modo polling, projetado para detectar a latência no nível do sistema operacional. Ele executa um loop ocupado (*busy loop*) com ou sem diferentes cargas de trabalho, coletando informações do contador de ciclos de tempo (*Time Stamp Counter - TSC*) e medindo o tempo de forma frequente durante o processo.

O teste é realizado em sistemas com múltiplos núcleos de CPU, ele pode medir a latência em diferentes núcleos para avaliar como a carga de trabalho e o agendamento de tarefas são distribuídos entre eles.

Comando utilizado:

```
oslat -b 512 -c 0,1,2,3 -w memmove -D 60 --json=resultado.json
```

- -b 512 Define o número de buckets ou faixas de latência que serão utilizados para classificar os resultados do teste.
- -c 0,1,2,3 Rodando nos núcleos 0, 1, 2 e 3.
- -w memmove Testando a operação memmove (cópia de memória).
- -D 60 Durante um período de 60 segundos.
- --json=resultado.json Arquivo com os resultados



```
str@str:~/rt-tests$ oslat -b 512 -c 0,1,2,3 -w memmove -D 60 --json=resultado.json
oslat V 2.20
Total runtime:      60 seconds
Thread priority:    default
CPU list:           0,1,2,3
CPU for main thread: 0
Workload:           memmove
Workload mem:       16 (KiB)
Preheat cores:      4

Pre-heat for 1 seconds...

Test starts...
Test completed.

Core:      0 1 2 3
CPU Freq:  2688 2688 2688 2688 (Mhz)
```

A frequência foi fixada em 2688 MHz para todos os núcleos durante o teste, garantindo consistência nos resultados.

Cada linha representa um intervalo de latência em microssegundos (us).

Os números representam a contagem de eventos de latência que ocorreram dentro daquele intervalo, para cada núcleo.

```
Core:      0 1 2 3
CPU Freq:  2688 2688 2688 2688 (Mhz)
001 (us):  87109627 404936425 403323344 405109583
002 (us):  2641 14599 14262 14200
003 (us):   32 123 119 111
004 (us):   29 114 104 118
005 (us):   16 61 60 46
006 (us):    9 18 23 22
007 (us):   26 97 55 106
008 (us):  249 1613 1528 1548
009 (us):  526 2412 2361 2400
010 (us):  283 1510 1338 1452
011 (us):  295 1286 1238 1271
012 (us):  237 950 983 960
013 (us):  216 919 934 994
014 (us):  160 619 599 571
015 (us):   85 341 320 286
016 (us):   49 213 139 163
017 (us):   21 99 97 76
018 (us):    8 82 68 60
019 (us):    9 38 54 50
020 (us):    5 23 38 36
021 (us):   11 29 35 18
022 (us):    9 37 29 28
023 (us):    6 27 33 20
024 (us):    2 32 19 38
025 (us):    4 34 19 17
026 (us):    5 31 24 17
027 (us):    4 17 17 16
028 (us):    5 11 19 22
029 (us):    1 11 15 13
030 (us):    4 5 17 11
```

Análise dos Resultados:

Percebe-se que muitas ocorrências ocorreram com baixa latência.

O núcleo 0 apresenta valores diferentes em comparação aos demais, isso pode ser explicado por ele também executar tarefas do sistema ou do próprio processo de controle do teste, o tempo máximo de latência que ocorreu neste núcleo foi 4003717 us.

```

485 (us): 0 0 0 0
486 (us): 0 0 0 0
487 (us): 0 1 1 0
488 (us): 0 0 0 0
489 (us): 0 0 0 1
490 (us): 0 0 0 1
491 (us): 0 0 0 0
492 (us): 0 0 1 0
493 (us): 0 0 0 0
494 (us): 0 1 0 0
495 (us): 0 0 0 0
496 (us): 0 2 0 1
497 (us): 0 0 0 0
498 (us): 0 0 0 1
499 (us): 0 0 1 1
500 (us): 0 0 0 0
501 (us): 0 0 0 1
502 (us): 0 0 0 0
503 (us): 0 0 0 0
504 (us): 0 0 0 0
505 (us): 0 0 0 0
506 (us): 1 0 1 0
507 (us): 1 1 0 0
508 (us): 0 0 0 1
509 (us): 0 0 0 0
510 (us): 0 0 0 0
511 (us): 0 0 0 0
512 (us): 80 312 283 329 (including overflows)
Minimum: 1 1 1 1 (us)
Average: 1.550 1.119 1.120 1.119 (us)
Maximum: 4003718 20506 35538 18108 (us)
Max-Min: 4003717 20505 35537 18107 (us)
Duration: 60.981 60.964 60.977 60.981 (sec)

```

Conforme o valor da latência vai aumentando, as ocorrências vão diminuindo.

Existe uma variação nos tempos entre os núcleos, mas em geral, os tempos são baixos. A variação é normal e pode ser causada pela distribuição do trabalho entre os núcleos.

As diferenças nos tempos de execução entre os núcleos podem ser atribuídas ao agendamento de tarefas pelo sistema operacional. Isso também pode refletir diferentes níveis de carga em cada núcleo durante o teste.

```

511 (us): 0 0 0 0
512 (us): 80 312 283 329 (including overflows)
Minimum: 1 1 1 1 (us)
Average: 1.550 1.119 1.120 1.119 (us)
Maximum: 4003718 20506 35538 18108 (us)
Max-Min: 4003717 20505 35537 18107 (us)
Duration: 60.981 60.964 60.977 60.981 (sec)

```

```

tr@str:~/rt-tests$

```

Teste ptsematest

O teste escolhido dentre os possíveis indicados pelo professor foi o teste ptsematest, ele mede a latência da comunicação interprocessual (IPC) em sistemas com prioridade de tempo real, utilizando mutex POSIX. Ela é uma ferramenta essencial para diagnosticar o desempenho e identificar potenciais problemas em sistemas com alta sensibilidade a atrasos, como sistemas embarcados.

Código usado:

```
sudo ptsematest -t2 -p99 -i1000 -l100000
```

Informações obtidas pelo código:

O ptsematest realiza medições de latência no bloqueio e desbloqueio dos mutexes nas threads que estão sendo rodadas com diferentes prioridades. Valores que são calculados:

- Latência Mínima(Min):
 - Menor Tempo registrado para adquirir o mutex;
- Latência Corrente (Cur):
 - Tempo da execução atual;
- Latência Média (Avg):
 - Média do tempo de execução calculada durante todos os ciclos
- Latência Máxima (Max):
 - Pior caso observado;
- Número de Ciclos:
 - Número total de iterações realizadas;
- Afinidade com CPUs:
 - Núcleo em que a thread foi executada.

Configurações escritas no código:

- 2 Threads:
 - -t2;
- Prioridade 99 e 98:
 - -p99;
- Intervalo entre execuções 1000us e 1500us:
 - -i1000;
- Rodará em 100000 ciclos:
 - -l100000.

```
1000000
str@str:~/rt-tests$ sudo ptsematest -t2 -p99 -i1000 -l100000
[sudo] password for str:
#0: ID9264, P99, CPU1, I1000; #1: ID9265, P99, CPU0, Cycles 100000
#2: ID9266, P98, CPU2, I1500; #3: ID9267, P98, CPU0, Cycles 78012
#1 -> #0, Min    0, Cur  344, Avg 2018, Max 999999997
#3 -> #2, Min    0, Cur  534, Avg 3086, Max 999999999  0
```

Análise dos Resultados:

Diante do que foi apresentado, é possível notar que a thread com maior prioridade (P99) apresenta latências em geral menores, o que é um comportamento esperado em sistemas em tempo real. Isso demonstra que o kernel RT está priorizando corretamente as threads. Portanto, ao olhar os valores da thread de prioridade mais baixa (no caso a de 98), fica claro que ela foi preemptada pela thread de prioridade 99, o que ocasionou no tempo de latência média e no geral ser maior.

Parte 2:

1) Você deve especificar o modelo de tarefa identificando (modelo básico visto em aula):

- a. Tempo de execução
- b. Tempo de Interferência. Pode ser adquirido utilizando a distribuição de prioridades.
- c. Período de ocorrência (pensando em periódico, esporádico ou aperiódico) no quesito tempo
- d. Deadline

2) Você deve analisar o Worst Case Execution Time e Worst Case Response Time das tarefas.

e. WCET

f) WCRT observado

Monitoramento de ABS:

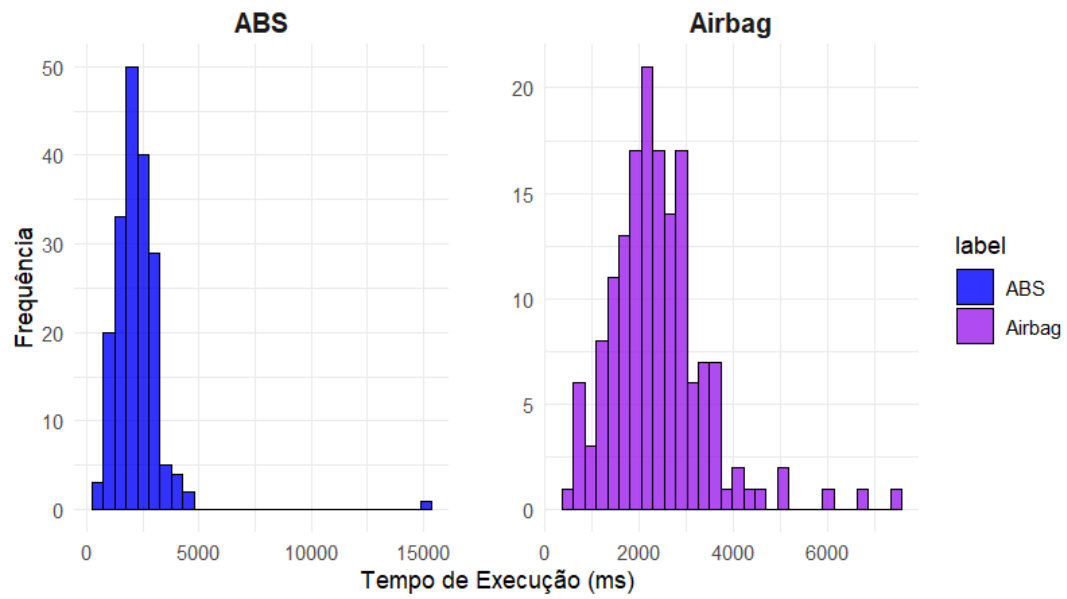
- a) 1906 us
- b) 448 us
- c) Esporádico
- d) 100 ms
- e) 14997 us
- f) 15035 us

Monitoramento Airbag:

- a) 1678 us
- b) 173 us
- c) Esporádico
- d) 100 ms
- e) 7434 us
- f) 7471 us

Histograma com amostra dos Tempos de Execuções:

Histogramas de Tempo de Execução por Label



Análise do WCRT e WCET na 5 com a análise temporal.

3) Indicar as características do hardware que podem afetar o tempo de execução e tempo de resposta.

As características do hardware que podem afetar o tempo de execução e resposta, em um sistema em tempo real são a arquitetura do computador, pois se for um sistema com múltiplos núcleos, pode processar as threads simultaneamente, aprimorando o tempo de execução, o processador, pois a depender da qualidade do processador, as instruções serão executadas em um tempo reduzido, prioridade de interrupções, pois se o sistema priorizar as threads de prioridade mais alta, vai tratá-los primeiro, assim minimizando atrasos em tarefas críticas e possivelmente atrasando atividades de prioridades menor, velocidade no acesso à memória, o acesso a memória ser o mais eficiente possível reduz o tempo necessário para salvar e recuperar valores durante troca de valores e a sobrecarga do sistema operacional, onde o overhead que está associado ao gerenciamento das threads, pode afetar negativamente o tempo de execução.

4) Utilize uma abordagem de medições identificando o HWM(100%) e também aplique o conceito de (m,k)-firme com 10 ativações de janela para cada tarefa. Usem como quantidade de medições de tempo pelo menos 20 amostras por tarefa. Também indique o fator skip entre perda de deadlines.

```
Tempo de execução do Airbag: 1170 us
Fator Skip: 0.00
Primeiros 10 valores, dentro do limite
Últimos 10 valores, dentro do limite
Tempo de execução do ABS: 900 us
Fator Skip: 0.00
Primeiros 10 valores, dentro do limite
Últimos 10 valores, dentro do limite
```

```
Fator Skip do Airbag: 0.00
Primeiros 10 valores, dentro do limite
Últimos 10 valores, dentro do limite
```

```
Fator Skip do ABS: 0.00
Primeiros 10 valores, dentro do limite
Últimos 10 valores, dentro do limite
```

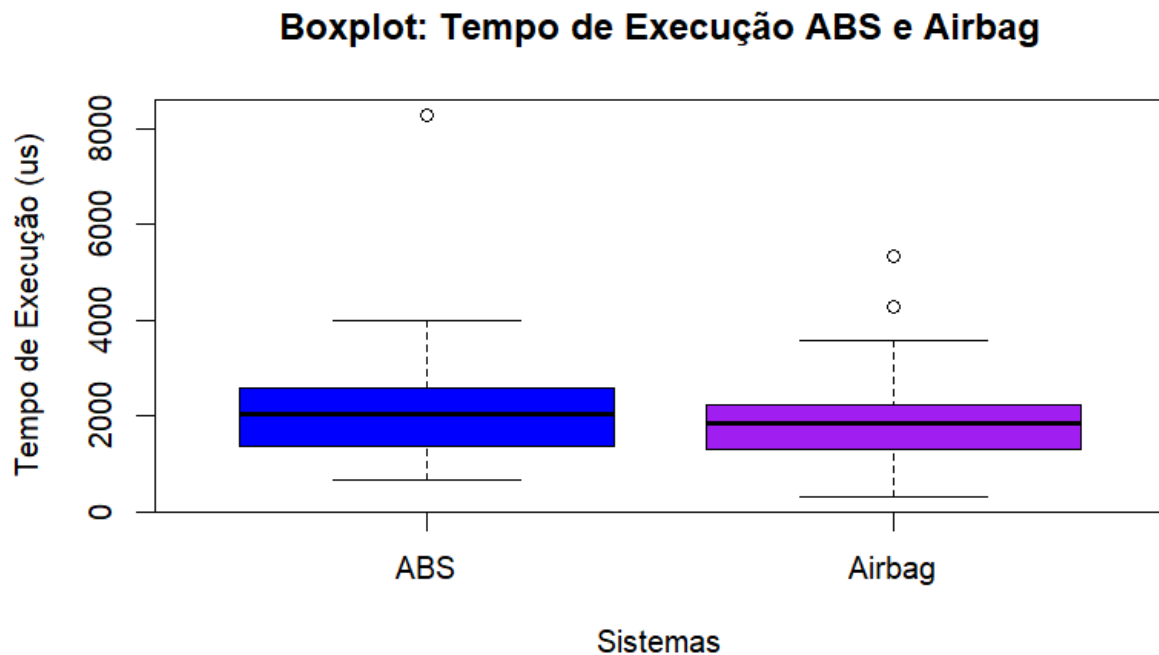
Como será demonstrado na análise temporal, o valor do fator skip em diversas amostras tem o valor de 0, portanto conclui-se que o programa está seguindo o requisito de HWM(100%). Tendo em vista que o valor de HMW utilizamos o mesmo valor do WCRT.

Task	Deadline	m	k	Fator Skip
ABS	100ms	8	10	Máximo de 2 falhas
Airbag	100ms	9	10	Máximo de 1 falha

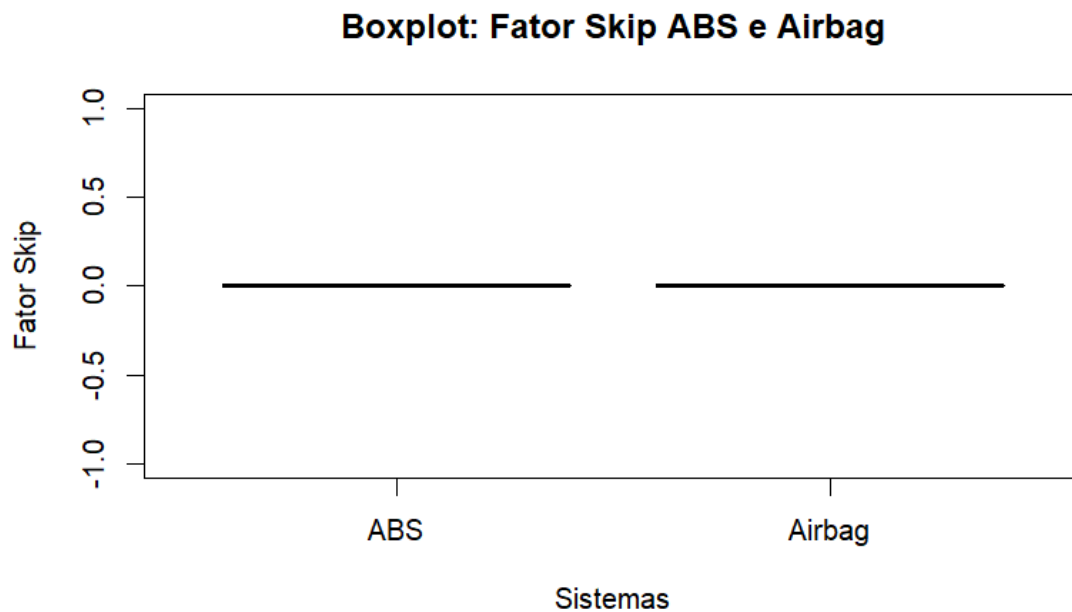
No programa foi feito um contador em cada task, este contador só adiciona o valor a si mesmo, quando entrar na condição que o sensor desta task foi detectado, assim a cada 20 ativações será feito a checagem do modelo (m, k)-firm, onde é feito a verificação primeiramente dos 10 primeiros valores, e depois dos últimos 10 valores, então é feito uma print dizendo se o tempo de execução da thread está dentro dos limites definidos do modelo.

5) Com o software, vocês deverão fazer uma análise temporal criteriosa do sistema desenvolvido.

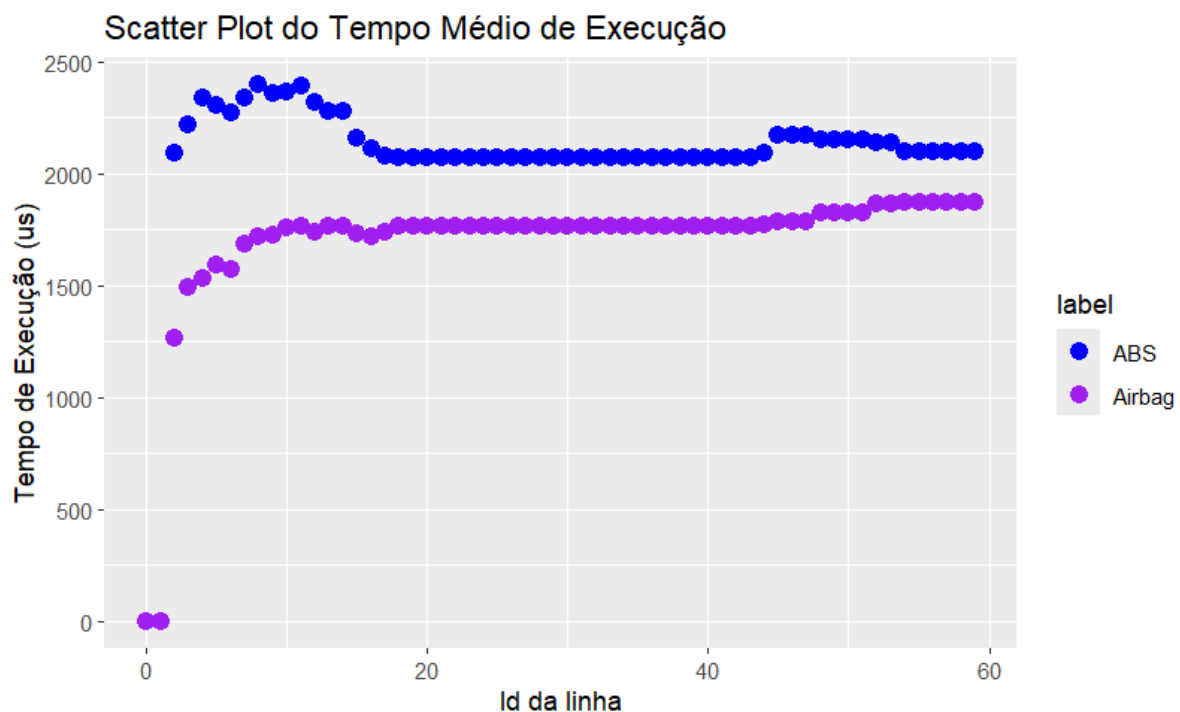
A análise temporal criteriosa, feita em R, do sistema foi feita a partir de uma amostra de dados, cerca de 500 linhas de dados com informações relevantes, retirado baseado nos prints feitos pelo programa. E como os valores são uma amostra, não necessariamente bateram com os valores apresentados na questão 2 e 3.



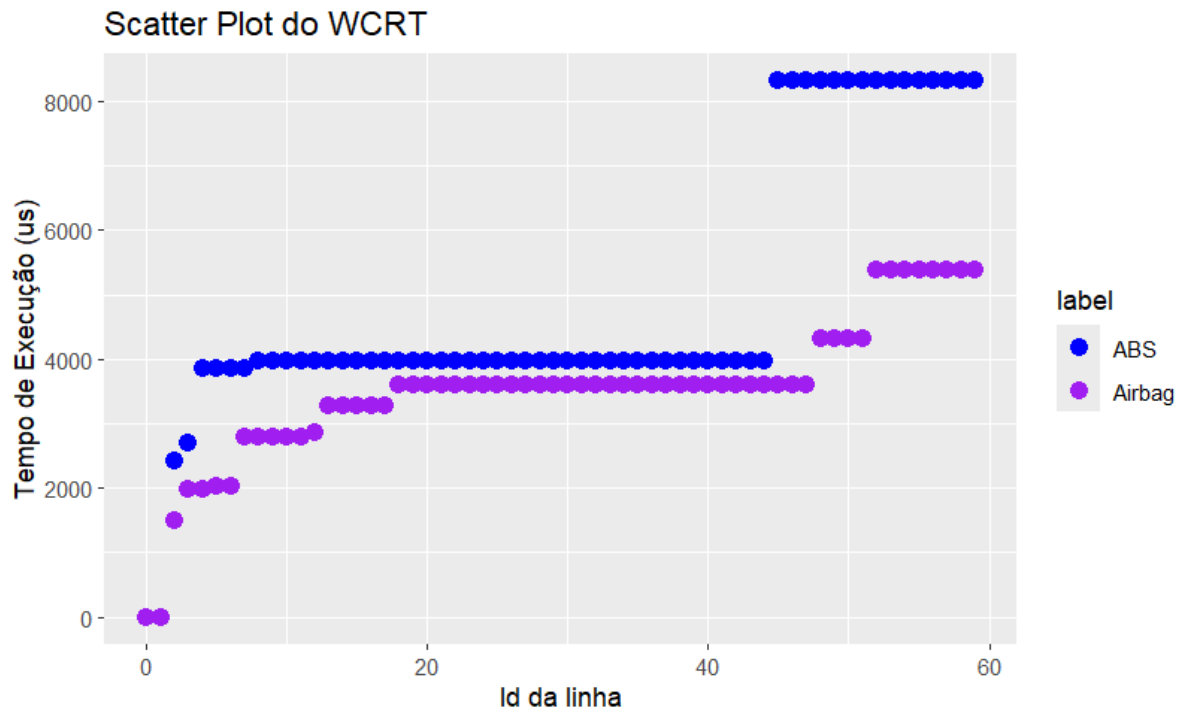
Como é possível notar na imagem acima, o tempo de execução da thread ABS e da thread Airbag são similares, com a presença de alguns outliers em maior quantidade no Airbag, e um outlier muito distante em ABS.



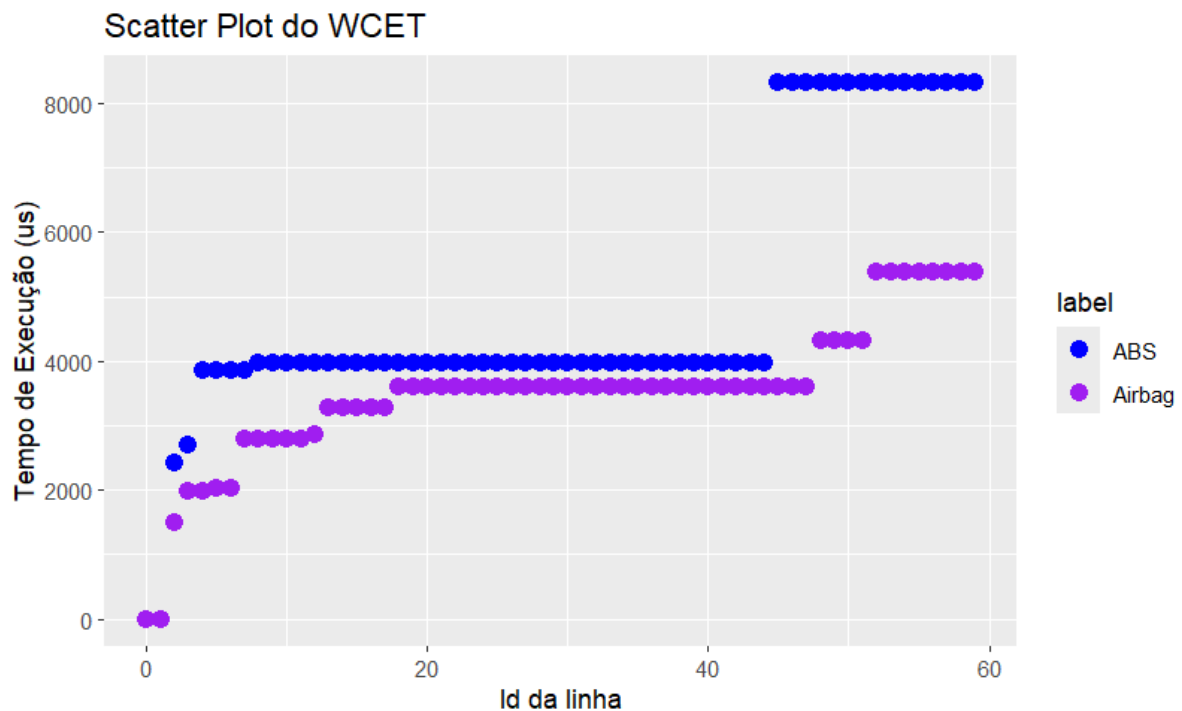
Diante da Imagem, é possível notar que nessa amostra, o Fator Skip ficou sempre no 0, ou seja, o número pré-definido de (m,k) firm foi cumprido.



Com base na imagem acima, tendo em vista que foi utilizado a variável “count” como id, o valor tempo médio de execução oscila, e depois de um período ele estabiliza tendo valores próximos, porém com o ABS tendo um valor um pouco superior ao do Airbag.



Diante da imagem acima, é possível notar que com o passar do tempo, de ficar ativando as teclas específicas, o valor de WCRT vai aumentando, porém estabilizando. Como nas outras visualizações gráficas, ABS e Airbag possuem valores similares com o ABS sendo um pouco superior, mas nesse caso no fim o ABS teve um caso que se distanciou do WCRT do Airbag, o que faz sentido, uma vez que Airbag possui uma prioridade mais alta.



O gráfico do WCET é basicamente idêntico ao WCRT, tendo em vista que como não há muitas threads, não ocorrem muitas interrupções.